DAY 3 Programs

NAME: AKASH J

Register No: 192324295

Program 1: Out of Boundary Paths

Aim:

To find the number of ways to move a ball out of an m x n grid boundary in exactly N steps from a starting cell.

Algorithm:

1. Use a 3D memoization array or a cache dictionary to store the number of ways for a given state `(r, c, steps)`.
2. If the current position `(r, c)` is out of the grid bounds, it counts as 1 valid path out. Return 1.
3. If `steps` reaches 0 and the ball is still inside, return 0.
4. Recursively call the function for all 4 adjacent directions (up, down, left, right) with `steps - 1`.
5. Return the sum of the results modulo `10^9 + 7` to prevent integer overflow.

Code:

```
def findPaths(m, n, N, i, j):
    memo = {}
    MOD = 10**9 + 7

    def dfs(r, c, steps):
        if r < 0 or r == m or c < 0 or c == n:
            return 1
        if steps == 0:
            return 0
        if (r, c, steps) in memo:
            return memo[(r, c, steps)]

        ways = 0
        for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:
            ways = (ways + dfs(r + dr, c + dc, steps - 1)) % MOD

        memo[(r, c, steps)] = ways
        return ways

    return dfs(i, j, N)
```

Input & Output:

- **Input 1:** `m=2, n=2, N=2, i=0, j=0`
  Output 1: `6`

- **Input 2:** `m=1, n=3, N=3, i=0, j=1`
  Output 2: `12`

Result:

The algorithm correctly computes the number of paths using Dynamic Programming with a time complexity of $O(m \times n \times N)$.

Program 2: House Robber II

Aim:

To find the maximum amount of money you can rob from houses arranged in a circle without triggering alarms.

Algorithm:

1. Handle base cases: if the array is empty, return 0; if it has one element, return that element.
2. Since houses are in a circle, the first and last houses are adjacent. Thus, we cannot rob both.
3. Create a helper function to solve the linear "House Robber" problem for a given sub-array.
4. Call the helper function twice: once for houses from index `0` to `n-2`, and once for index `1` to `n-1`.
5. Return the maximum of the two scenarios.

Code:

```
def rob(nums):
    if not nums: return 0
    if len(nums) == 1: return nums[0]

    def rob_linear(houses):
        rob1, rob2 = 0, 0
        for money in houses:
            new_rob = max(rob1 + money, rob2)
            rob1 = rob2
            rob2 = new_rob
        return rob2

    return max(rob_linear(nums[:-1]), rob_linear(nums[1:]))
```

Input & Output:

- **Input 1:** `nums = [2, 3, 2]`
  Output 1: `3`
- **Input 2:** `nums = [1, 2, 3, 1]`
  Output 2: `4`

Result:

The dynamic programming approach runs in $O(n)$ time and $O(1)$ space.

Program 3: Climbing Stairs

Aim:

To find the number of distinct ways to climb to the top of an n-step staircase, taking 1 or 2 steps at a time.

Algorithm:

1. Recognize that to reach step `i`, one must either step from `i-1` or `i-2`. This models the Fibonacci sequence.
2. Initialize two variables, `prev1 = 1` and `prev2 = 1`, representing the ways to reach the 1st and 0th step.
3. Iterate from step 2 to n.
4. In each iteration, the current step ways equal `prev1 + prev2`. Update variables accordingly.
5. Return the final computed ways.

Code:

```
def climbStairs(n):
    if n <= 1:
        return 1
    prev1, prev2 = 1, 1
    for i in range(2, n + 1):
        current = prev1 + prev2
        prev2 = prev1
        prev1 = current
    return prev1
```

Input & Output:

- **Input 1:** `n = 4`
  Output 1: 5
- **Input 2:** `n = 3`
  Output 2: 3

Result:

The ways to climb the stairs are found successfully using $O(n)$ time complexity and $O(1)$ space complexity.

Program 4: Unique Paths

Aim:

To calculate the number of possible unique paths a robot can take from the top-left to the bottom-right corner of an m x n grid.

Algorithm:

1. Create a 1D DP array dp of size n, initialized with 1s, representing the first row.
2. Iterate through the rows from 1 to `m-1`.
3. For each column from 1 to `n-1`, update `dp[j]` to be `dp[j] + dp[j-1]`.
4. This iteratively calculates the paths for each cell based on the cell above it (`dp[j]`) and the cell to its left (`dp[j-1]`).
5. Return `dp[-1]`.

Code:

```
def uniquePaths(m, n):
    dp = [1] * n
    for i in range(1, m):
        for j in range(1, n):
            dp[j] += dp[j - 1]
    return dp[-1]
```

Input & Output:

- **Input 1:** `m=7, n=3`
  Output 1: `28`
- **Input 2:** `m=3, n=2`
  Output 2: `3`

Result:

The total unique paths are calculated with $O(m \times n)$ time complexity and $O(n)$ space complexity.

Program 5: Positions of Large Groups

Aim:

To identify intervals of every large group (3 or more consecutive identical characters) in a string.

Algorithm:

1. Initialize a list `result` to store the intervals, and a variable `start = 0` to mark the beginning of a group.
2. Iterate through the string with index `i` from 0 to `len(s) - 1`.
3. Check if we are at the last character, or if the current character `s[i]` is different from the next character `s[i+1]`.
4. If a group ends, check its length: `i - start + 1`.
5. If length ≥ 3, append `[start, i]` to the result. Update `start = i + 1`.

Code:

```
def largeGroupPositions(s):
    result = []
    start = 0
    for i in range(len(s)):
        if i == len(s) - 1 or s[i] != s[i + 1]:
            if i - start + 1 >= 3:
                result.append([start, i])
            start = i + 1
    return result
```

Input & Output:

- **Input 1:** `s = "abbxxxxzzy"`
  Output 1: `[[3,6]]`
- **Input 2:** `s = "abc"`
  Output 2: `[]`

Result:

The intervals are correctly extracted in a single pass resulting in $O(n)$ time complexity.

Program 6: Game of Life

Aim:

To compute the next generation of the cellular automaton Game of Life in-place.

Algorithm:

1. To do this in-place, introduce intermediate states: 2 (was live, now dead) and 3 (was dead, now live).
2. Loop over the grid. For each cell, count its live neighbors (checking for values 1 or 2).
3. Apply rules: If a live cell has < 2 or > 3 live neighbors, it becomes 2. If a dead cell has exactly 3 live neighbors, it becomes 3.
4. Loop over the grid again to finalize states: Change 2 to `0`, and 3 to 1.

Code:

```
def gameOfLife(board):
    m, n = len(board), len(board[0])

    def count_live(r, c):
        live = 0
        for dr in [-1, 0, 1]:
            for dc in [-1, 0, 1]:
                if dr == 0 and dc == 0: continue
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and abs(board[nr][nc])
== 1:
                    live += 1
        return live

    for r in range(m):
        for c in range(n):
            lives = count_live(r, c)
            if board[r][c] == 1 and (lives < 2 or lives > 3):
                board[r][c] = -1 # -1 means live -> dead
            if board[r][c] == 0 and lives == 3:
                board[r][c] = 2  # 2 means dead -> live

    for r in range(m):
        for c in range(n):
            if board[r][c] == -1: board[r][c] = 0
            elif board[r][c] == 2: board[r][c] = 1
    return board
```

Input & Output:

- **Input 1:** `board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]`
  Output 1: `[[0,0,0],[1,0,1],[0,1,1],[0,1,0]]`
- **Input 2:** `board = [[1,1],[1,0]]`
  Output 2: `[[1,1],[1,1]]`

Result:

The state changes successfully in-place with $O(m \times n)$ time complexity.

Program 7: Champagne Tower

Aim:

To determine how full a specific glass is after a given amount of champagne is poured down a pyramid of glasses.

Algorithm:

1. Use Dynamic Programming. Initialize a 2D array dp where `dp[i][j]` holds the liquid passing through glass `j` in row `i`.
2. Set `dp[0][0] = poured`.
3. Iterate through rows `i` from 0 to `query_row`.
4. Iterate through glasses `j` in row `i`. If `dp[i][j] > 1.0`, the excess flows down.
5. Add `(dp[i][j] - 1.0) / 2.0` to `dp[i+1][j]` (left child) and `dp[i+1][j+1]` (right child).
6. The final answer for the query glass is `min(1.0, dp[query_row][query_glass])` because a glass cannot hold more than 1 cup.

Code:

```
def champagneTower(poured, query_row, query_glass):
    dp = [[0.0] * k for k in range(1, query_row + 2)]
    dp[0][0] = poured

    for i in range(query_row):
        for j in range(len(dp[i])):
            if dp[i][j] > 1:
                excess = (dp[i][j] - 1.0) / 2.0
                dp[i+1][j] += excess
                dp[i+1][j+1] += excess

    return min(1.0, dp[query_row][query_glass])
```

Input & Output:

- **Input 1:** `poured = 1, query_row = 1, query_glass = 1`
  Output 1: `0.00000`
- **Input 2:** `poured = 2, query_row = 1, query_glass = 1`
  Output 2: `0.50000`

Result:

The champagne amount is simulated effectively resulting in an $O(R^2)$ time complexity where R is the query_row.